

AI-based Tuberculosis (TB) Screening from Chest X-rays

Domain: Healthcare | AI/ML Based Proof of Concept (POC)

1. Problem Statement

Pakistan WHO ki high-TB-burden countries ki list mein shamil hai. Rural aur semi-urban areas mein radiologists aur TB specialists ki shadeed kami hai, jiske wajah se diagnosis mein delay hota hai.

2. Proposed AI/ML Solution

- Patient ka chest X-ray image upload kiya jayega API ke through.
- Pre-trained deep learning model image analyze karke TB ke signs detect karega.
- System risk score (Low/Medium/High) ke sath result return karega proper JSON response mein.
- High-risk cases DB mein flag ho ke referral ke liye admin dashboard mein dikhengi.

3. Tools & Technologies

TensorFlow/PyTorch (CNN), OpenCV (preprocessing), FastAPI (backend), PostgreSQL (records), Streamlit (demo interface)

4. Dataset

Shenzhen & Montgomery TB Chest X-ray Datasets (Kaggle/NIH par publicly available)

5. Feasibility for POC

High — publicly available labeled dataset hai aur binary classification model relatively seedha implement hota hai.

6. Expected Impact

Rural areas mein bina specialist ke bhi initial screening possible hogi, early detection se TB ka phailav control hoga.

Technical Defense — Sir Q&A Preparation

7. Kya Use Karoon Gi (My Tech Stack) & Kyun

- Pre-trained CNN (transfer learning, ResNet18) — kam data mein bhi achi accuracy deta hai.
- FastAPI + async endpoints — bade image files handle karne ke liye async better hai.
- PostgreSQL — patient/prediction records securely store karne ke liye, exception handling ke saath.
- JWT authentication — sensitive health data ke liye access control zaroori hai.

8. Kya Use NAHI Karoon Gi & Kyun (Rejected Alternatives)

- Real patient data — privacy/ethics ka masla hai, isliye sirf anonymized public dataset use kiya.
- Final diagnosis ka claim — model sirf 'screening/risk assessment tool' hai, doctor ka replacement nahi.
- Multi-disease detection ek sath — scope sirf TB tak limited rakha for POC clarity.

9. Kaise Implement Karoon Gi (Step-by-Step)

- Step 1: Dataset load aur preprocess (resize, normalize).
- Step 2: Pre-trained model fine-tune binary classification ke liye.
- Step 3: FastAPI endpoint jo X-ray accept kare (form-data) aur prediction return kare.
- Step 4: Result + confidence PostgreSQL mein log, try-except ke sath.
- Step 5: Streamlit demo UI jahan doctor image upload kare aur risk-level dekhe.

10. Possible Issues / Challenges & Mitigation

- False positives/negatives ka risk — isliye 'risk score' terminology use ki.
- Medical data privacy — sirf public dataset use kiya.
- Model bias agar dataset limited demographics se ho.

11. Anticipated Questions from Sir & Answers

Q: Yeh model doctor ka replacement hai?

A: Bilkul nahi, sirf screening/assistance tool hai; final diagnosis hamesha doctor karega.

Q: Agar model galat prediction de to?

A: Confidence score bhi show hoga, high-risk cases mandatory doctor referral ke sath flag honge.

Q: Data privacy kaise ensure karogi?

A: Sirf public anonymized dataset use kiya; real deployment mein JWT aur encrypted storage zaroori hogi.

Project Architecture & Folder Structure

12. Folder Structure (Sir ke pattern par based)

```
tb_screening_system/
├── src/
│   ├── screenings/
│   │   ├── __init__.py
│   │   ├── models.py          # Screening table (id, patient_ref, xray_path, risk_level, created_at)
│   │   ├── schemas.py        # ScreeningCreate, ScreeningResponse
│   │   ├── router.py          # POST /screenings/analyze, GET /screenings/{id}
│   │   ├── services.py
│   │   ├── utils.py           # image preprocessing, model inference
│   │   └── users/
│   │       ├── __init__.py
│   │       ├── models.py      # doctor/admin login (JWT protected)
│   │       ├── schemas.py
│   │       ├── router.py
│   │       └── services.py
│   ├── database.py
│   ├── settings.py
│   ├── main.py
│   └── ml_model/
│       └── tb_xray_model.h5
├── uploaded_xrays/
├── logs/
│   └── app.log
├── streamlit_app.py
├── requirements.txt
├── .env
├── .gitignore
├── notes.ipynb
└── README.md
```

13. Har File Mein Kya Likha Hoga

- **models.py** — SQLAlchemy table definition (e.g. screenings table: id, columns, timestamps). Yeh DB structure define karta hai.
- **schemas.py** — Pydantic models jo request body aur response ka shape/validation define karte hain (e.g. Create schema, Response schema).
- **router.py** — FastAPI endpoints (@router.get, @router.post, etc.) yahan define hote hain — yeh sirf request receive karke services ko call karta hai.
- **services.py** — Business logic yahan hoti hai: DB query build karna, execute karna, result process karna, phir router ko return karna.
- **utils.py** — Helper/reusable functions (e.g. image preprocessing, prediction formatting, common validation) jo ek se zyada jagah use hote hain.
- **database.py** — SQLAlchemy engine, SessionLocal, aur Base declaration — DB connection setup yahan hota hai.
- **settings.py** — .env se DATABASE_URL, API keys, model paths load karta hai (Pydantic Settings/BaseSettings).
- **main.py** — FastAPI app instance banata hai, sab routers include karta hai (app.include_router), aur entry point hai (uvicorn isi ko run karta hai).

14. Data Flow — Sab Kaise Connect Hota Hai

- User Streamlit app se chest X-ray input deta hai (image/data) → request FastAPI ke router.py tak jati hai.
- router.py request ko services.py ko forward karta hai (router sirf routing karta hai, logic nahi rakhta).
- services.py business logic run karta hai: model se prediction lena, ya database query karna (SQLAlchemy session use karke).
- Agar DB involved hai, services.py database.py ke SessionLocal se connect hota hai aur models.py ki table mein data insert/fetch karta hai.
- Result schemas.py ke response model ke mutabiq format hoke wapis router se Streamlit/client ko JSON response mein jata hai.
- Har step par logging aur try-except exception handling hoti hai taake error properly track ho aur 200/500 status code sahi return ho.

15. pgAdmin Mein Kya Data Show Hoga

- pgAdmin mein connect karne ke baad, database ke andar **screenings** table dikhega jab main.py chalega aur SQLAlchemy models create_all() run karega.
- Table ke columns: id, patient_ref, xray_path, risk_level, confidence_score, created_at.
- Query Tool se 'SELECT * FROM screenings;' likh ke run karne par saara stored data (predictions/records) rows ke form mein dikhega.
- Jaise jaise API calls (POST requests) hongi, naye rows real-time mein is table mein add hote jayenge — yeh live demo mein sir ko dikhaya ja sakta hai.

16. Streamlit Interface Mein Kya Dikhega

- Streamlit app browser mein khulega (streamlit run streamlit_app.py) — ek simple web page jismein title aur upload/input section hoga.
- User chest X-ray image upload/enter karega → 'Predict/Submit' button dabayega.
- Streamlit background mein FastAPI ke endpoint ko requests.post() se call karega (jaisa maine API integration tasks mein practice ki).
- Result screen par display hoga: TB risk ka prediction/output, confidence score, aur agar history feature hai to past records ki list bhi.

17. uvicorn Chalane Par Kaunsi APIs Show Hongi (/docs)

- uvicorn src.main:app --reload chalane ke baad, browser mein http://127.0.0.1:8000/docs kholne par Swagger UI dikhega.
- Wahan '/screenings' prefix ke andar sab endpoints list honge: POST /screenings/analyze (X-ray -> risk result), GET /screenings/{id}, GET /health.
- Har endpoint 'Try it out' se directly test kiya ja sakta hai — request body dal ke 'Execute' karne par response (200 success ya error) turant dikhega.
- /redoc par same APIs ka clean documentation view bhi milta hai.

18. Additional Anticipated Questions from Sir

Q: JWT kahan use hoga is system mein?

A: Doctor/admin login endpoints par — sirf authenticated users hi patient screening records access/create kar sakenge, jaisa maine users module mein implement kiya tha.

Q: Agar do models compare karne hon (ResNet vs MobileNet) to?

A: Dono ko train karke accuracy/F1-score compare karoon gi, aur better performing model ko production ke liye choose karoon gi — yeh evaluation notebook mein document karoon gi.

Q: Real deployment mein hosting kahan hogi?

A: POC ke liye local/demo hi hai; real deployment ke liye cloud (AWS/Azure) ya hospital ka apna server use ho sakta hai, jo future scope hai.